

Projet Web - Dodge the Blocks

URL : <https://etud.insa-toulouse.fr/~saborit-lope/>

Nom	N° étudiant
DRIDI Mohamed-Amine	0510004500571
LABASSA-DESTOUESSE Nathan	0510004500580
IVAKHNIUK Nazarii	0510004300020
SABORIT-LOPEZ Iker	0510004300152

Structure du projet

Fichiers racine : **index.html**, **README.md**, image **discheme.jpeg**. Dossiers : **scripts/** (moteur JS), **styles/** (CSS).

Dossier	Fichiers principaux (exemples)
scripts/	main.js, Game.js, Pxlayer.js, ObstacleManager.js, ColisionDetector.js, DOMRender.js, Templates.js, ScoreManager.js, InputManager.js, KeyboardInput.js, Constants.js
styles/	style.css

Aspects obligatoires

Exigence	Fichier(s) et lignes	Choix / justification (très bref)
Zone de jeu	Templates.js l.30-39; DOMRender.js l.90-111 + l.113-123 + l.136-144; style.css l.102-114	Le "game-area" est un <i>div</i> (position:relative). Les entités sont en absolute pour faciliter placement, collisions et suppression.
Joueur contrôlé (flèches/WASD)	Constants.js l.5-26; KeyboardInput.js l.1-41; InputManager.js l.1-12 ; player.js l.31-85 + l.87-114 + l.266-285	Mappage des touches isolé (INPUT_BINDINGS) et état d'actions centralisé : ajout de touches simple, code plus lisible.
Obstacles qui tombent	ObstacleManager.js l.31-131	Tableau obstacles[] : itération par index et suppression sûre en parcourant de la fin vers le début.
Détection de collision (game over)	ColisionDetector.js l.20-35 + l. 43-51; game.js l. 145-156	AABB (rectangles) : calcul très rapide, adapté aux <i>div</i> CSS ; retourne l'obstacle touché pour déclencher gameOver().
Score qui augmente avec le temps	game.js l. 134-135; ScoreManager.js l.14-39; DOMRender.js l.125-128	Incrémentation avec deltaTime : score indépendant du FPS ; affichage via DOMRender.drawScore().
Menu de démarrage	Templates.js l.1-28; DOMRender.js l. 22-73 ; main.js l.32-57	Menus générés par gabarits puis injectés dans le DOM : centralise l'UI et évite de maintenir du HTML dupliqué. Utilisation de Flexbox avec flex-direction : column pour centrer les éléments.
Bouton Rejouer	Templates.js l.51-53 + l. 64-67 (pause); main.js l.66-72 + l.83-89; game.js l.44-65 (init())	Relance startGame() ; init() réinitialise score, obstacles, bullets et position joueur.
Classement (session)	Templates.js l.11-14 (leaderboard UI); main.js l.106-114 (updateLeaderboard rendering); ScoreManager.js l.41-50 (localStorage save/get)	(<ul id="scoreboard-list">). Classement local persistant via localStorage (top 5) ; meilleur score également conservé via cookie (SameSite=Lax) ; permet d'afficher le leaderboard entre sessions.

Attendus pédagogiques (JS) - éléments observables

Point demandé	Implémentation (fichier/lignes) + justification
1) String / chaînes	ScoreManager.js l.56-57 (template string cookie); main.js l.39-42 (trim)
2) Fonctions	startGame() main.js l.100-104; Game.update() game.js l.122-135; ObstacleManager.applyMarkovChain() l.93-107; Player.move() l.87-114

3) if/else	main.js l.37-54 (pseudo + difficulté); ObstacleManager.js l.68-82 (level==4 + rebound); Player.js l.45-74 (dash logic); CollisionDetector.js l.43-50
4) Boucles	Player.updateBullets() Player.js l.208-236; Game.handleCollisions() game.js l.162-191; CollisionDetection.checkPlayerCollision() CollisionDetector.js l.20-35; DOMRender.initializeContainers() DOMRender.js l.46-60; main loop: requestAnimationFrame in game.js l.86-116

Point demandé	Implémentation (fichier/lignes) + justification
5) Tableaux	ObstacleManager.js l.8 + l.54-90 (obstacles[]); Player.js l.24 + l.211-235 (bullets[]); InputManager.js l.2-12 (sources[]); leaderboard array: ScoreManager.js l.42-46 (scores[])
6) Validation JS	main.js l.39-42
7) Objets + prototypes	Classes ES6 : Game, Player, ObstacleManager, ScoreManager, DOMRender, InputManager, KeyboardInput, CollisionDetection. Choix : séparation des responsabilités et code réutilisable.
8) DOM (ajout/suppression/màj)	DOMRender.renderTemplate() l.90-111; DOMRender.updateElementPosition() l.113-123; DOMRender.removeElement() l.130-133; ScoreManager.updateDisplay() l.35-39; pause score DOM: game.js l.197-205; game over DOM: game.js l.236-250

Extensions (difficulté, trajectoires, tir)

Niveaux : **Templates.js** : l.16-24 (select difficulté), **main.js** : l.44-54 (conversion difficulté → level),
ObstacleManager.js : l.158-178 (getLevel), **player.js** : l.286-291 (getLevel);
Trajectoires (drift) : **ObstacleManager.js** : l.93-107 (applyMarkovChain), l.67-74 (application du drift dans update);
Dash + tir + collisions balle/obstacle : Dash : **player.js** : l.45-74 + l.116-124; Tir : **player.js** : l.126-171 (shoot) + l.208-236 (update bullets); Collision balles/obstacles + +50 points : **game.js** : l.162-191 (collision) + l.186-187 (+50)

Description du projet

Le projet est un jeu de survie dans lequel des obstacles circulaires rouges apparaissent en haut de l'écran et se déplacent vers le bas. Leur mouvement peut inclure des déplacements verticaux, ainsi que latéraux (gauche-droite) déclenchés de manière probabiliste à l'aide d'une chaîne de Markov. L'activation de ce comportement dépend du niveau de difficulté.

Le mouvement latéral des obstacles n'est pas constant et est évalué à chaque rafraîchissement. La probabilité d'activation est d'environ 1 % au niveau 1, 1,5 % au niveau 2, 2 % au niveau 3 et 2,5 % au niveau 4. Le déplacement latéral est donc rare aux niveaux faibles et devient progressivement plus fréquent lorsque la difficulté augmente. Ces valeurs sont implémentées directement dans le code sous forme de probabilités par frame.

La difficulté du jeu évolue selon le niveau choisi (*Easy*, *Mid*, *Scary* et *Don't*). En *Easy*, les obstacles apparaissent lentement et suivent des trajectoires majoritairement verticales. Le mode *Mid* augmente la fréquence d'apparition et introduit des changements de direction horizontale plus réguliers. En *Scary*, les obstacles sont générés plus rapidement et leurs déplacements deviennent nettement plus imprévisibles. Enfin, le mode *Don't* correspond au niveau le plus extrême : les obstacles apparaissent presque en continu et bénéficient d'un déplacement latéral fortement accentué, augmentant fortement la densité d'obstacles à l'écran sans accélérer excessivement leur chute verticale.

Le joueur est représenté par un élément HTML (div) de 30 × 30 pixels contrôlé par l'utilisateur (Arrows / WASD (dans un clavier US)). Il dispose d'une capacité de dash (LShift) qui augmente temporairement sa vitesse et verrouille la direction du mouvement pendant 180 ms. Le joueur peut également tirer des projectiles vers le haut de l'écran (Space), avec un temps de recharge de 300 ms, valeur vérifiable dans le code. En raison des déplacements latéraux des obstacles, il devient plus difficile de les atteindre lorsqu'ils se déplacent horizontalement. Le tir est donc généralement utilisé en maintenant la touche de tir enfoncée afin d'augmenter les chances de toucher une cible.

Le score du joueur augmente continuellement avec le temps de survie. Il est possible d'accélérer cette progression en détruisant des obstacles, chaque ennemi éliminé rapportant environ 50 points, soit l'équivalent d'une demi-dizaine de secondes de survie. Le tir constitue ainsi une mécanique optionnelle mais stratégique, apportant une récompense significative au prix d'un risque supplémentaire.

La vitesse du joueur évolue également selon le niveau. Aux niveaux 1 à 3, elle augmente progressivement. Au niveau 4, elle est volontairement réduite à une valeur proche de celle du niveau 2 afin d'améliorer la précision du contrôle, compte tenu du nombre élevé d'obstacles présents à l'écran et du risque qu'une vitesse excessive rende le jeu difficilement maîtrisable.